

Do que é feito um agente baseado em LLM?

Encontro 03 · Sistemas Multiagentes / Agentic AI

Do encontro anterior

objetivo + estado + ações permitidas

↓

decisão

O CampusBot já escolhia parte da trajetória em tempo de execução.

Mas quem sustenta a trajetória depois da primeira escolha?

Uma boa resposta já basta?

Execute uma resposta isolada

```
uv run python praticas/encontro-03/01_uma_resposta.py
```

Leia a saída. Ainda não explique a arquitetura.

Uma pessoa escreve

“O projetor não funciona e a apresentação começa em breve.”

entrada → LLM ou função → resposta

O que falta para ela continuar ajudando?

- Qual objetivo orientou a resposta?
- Que informação ainda precisa ser obtida?
- Quem escolhe o próximo passo?
- Como o sistema sabe que terminou?

Uma possível análise

Resposta isolada

Pode ser útil e bem escrita.

Trajetória

Ainda não declara estado, próxima decisão ou parada.

O problema não é a qualidade do texto. É a responsabilidade de controle que não está visível.

Uma LLM pode responder.

Isso não basta, por si só, para organizar uma sequência de decisões.

Agora faça a conversa virar um ciclo

Antes de executar, preveja:

Qual deve ser a primeira escolha para o caso do projetor?

```
uv run python praticas/encontro-03/02_ciclo_controlado.py
```

Percepção inicial

0 projetor não funciona e a apresentação começa em breve.

```
estado = { local: ?, problema: projetor, urgente: sim }
```

Primeira decisão

ação: perguntar_local
motivo: o local é necessário para orientar o próximo passo

“Em qual local isso está acontecendo?”

O interlocutor responde

É no Lab 4.

Isso é uma **nova percepção**.

O estado muda

```
antes  local = ?  
depois local = Lab 4
```

A segunda decisão poderá usar algo que não estava disponível na primeira.

Segunda decisão

ação: `orientar_e_encerrar`

motivo: o agente reuniu a informação necessária,
mas seu limite é conversacional

Por que ele não abre um chamado?

Porque ele ainda não possui essa capacidade

“Chamado aberto.”

seria apenas texto, não uma ação verificada no ambiente.

Neste encontro: ciclo e responsabilidades. No próximo: ferramentas e ação externa.

O ciclo que acabamos de observar

objetivo

↓

percepção → estado → decisão → próximo passo

↑

interlocutor ← controlador ← validação / parada

Objetivo e limites

ajudar com o problema do projetor
sem inventar uma ação externa

O que o sistema tenta alcançar — e o que não pode alegar fazer?

Percepção

mensagem inicial
nova resposta da pessoa

O que acabou de ficar conhecido?

Estado de execução

```
local = Lab 4  
problema = projetor  
urgente = sim
```

O que o sistema precisa conservar para a próxima decisão?

Modelo de decisão

objetivo + percepção + estado
↓
próximo passo

Ele pode ser uma LLM, uma regra ou outro mecanismo.

Diante do que sei, o que faz sentido fazer agora?

Ações permitidas

perguntar o local
orientar e encerrar

O conjunto é pequeno de propósito.

| O que este agente pode realmente escolher?

Controlador do ciclo

entrega informações
→ valida a decisão
→ obtém nova percepção ou encerra

Quem mantém a execução coerente entre uma decisão e outra?

Condição de parada

```
concluiu = verdadeiro
```

ou

```
limite de passos atingido
```

Quando o sistema deve deixar de agir?

Componentes não são sinônimos

LLM

pode propor uma decisão

Controlador

organiza o ciclo e seus limites

Uma biblioteca pode juntar responsabilidades; isso não apaga as perguntas arquiteturais.

E se uma responsabilidade falhar?

Desligue o estado de propósito

Preveja o passo seguinte depois de a pessoa informar “É no Lab 4”.

```
uv run python praticas/encontro-03/03_diagnosticar_o_ciclo.py --sem-estado
```

Registro da versão defeituosa

NOVA PERCEPÇÃO: É no Lab 4.
ESTADO ATUAL: local = ?

PASSO 2
DECISÃO: perguntar_local

O agente pergunta de novo.

A LLM “esqueceu”?

Análise do diagnóstico

Não precisamos atribuir o problema genericamente ao modelo.

percepção ocorreu



estado não foi atualizado



decisão seguinte não recebeu a informação

A falha está na passagem entre percepção e estado.

Agora interrompa um ciclo que funciona

```
uv run python praticas/encontro-03/03_diagnosticar_o_ciclo.py --limite 1
```

Ele deveria ter terminado depois de uma única decisão?

Registro da outra falha

PASSO 1: perguntar local
NOVA PERCEPÇÃO: É no Lab 4.

PARADA DE SEGURANÇA: limite de passos atingido

O agente recebeu a informação necessária, mas não pôde usá-la.

Limites de passos não demonstram inteligência

Eles são um mecanismo de controle.

sem limite → o ciclo pode repetir indefinidamente
limite inadequado → o ciclo pode parar cedo demais

Estado não é, ainda, “memória” em sentido completo

Aqui, estado é a informação de trabalho desta execução.

```
local informado nesta conversa
```

Contexto, estado e memória serão investigados com mais precisão no Encontro 05.

Onde entra uma LLM real?

Uma LLM sob restrições

```
uv run python praticas/encontro-03/02_ciclo_controlado.py --online
```

Ela recebe:

```
objetivo + observação + estado + ações conversacionais
```

e devolve uma decisão estruturada.

O controlador ainda verifica a saída

```
ação pertence ao conjunto permitido?  
mensagem é texto?  
decisão declara parada?  
a combinação ação / parada é coerente?
```

O modelo propõe. A arquitetura aceita, recusa ou continua.

O que uma LLM não ganha automaticamente

acesso a e-mail

acesso a banco de dados

acesso a terminal

acesso a equipamentos ou sistemas
institucionais

No modo offline, a LLM é simulada

```
if local ausente → perguntar local  
if local presente → orientar e encerrar
```

Isso não transforma a lição em “if versus LLM”.

A arquitetura continua visível porque o problema é organizar informações, limites e continuidade.

Reconstrua o percurso

uma resposta

→ não deixa clara uma trajetória

objetivo + percepção + estado + decisão limitada

→ permite escolher o próximo passo

controlador + validação + parada

→ produz um ciclo controlado

Duas falhas, duas responsabilidades

Sem estado

repete uma pergunta apesar de receber a resposta.

Limite baixo

encerra antes de usar a nova percepção.

Uma LLM pode participar da decisão.

Um agente organiza decisões, informações, limites e consequências ao longo de uma trajetória.

O que ainda falta ao CampusBot?

- conversar ✓
- manter estado ✓
- controlar o ciclo ✓
- agir no ambiente ?

Missão autônoma — Localize a responsabilidade que falta

Abra:

```
uv run python praticas/encontro-03/atividade_autonoma.py
```

Antes de modificar código, registre:

```
objetivo · percepção · estado · próximo passo · parada · limite atual
```

Depois, compare as duas execuções

```
uv run python praticas/encontro-03/03_diagnosticar_o_ciclo.py --sem-estado
```

Registre:

1. a responsabilidade que falhou;
2. uma evidência no terminal;
3. a menor mudança arquitetural que a repararia.

**Que ferramenta ou capacidade externa o
CampusBot precisaria ter
para fazer mais do que responder?**